TRƯỜNG ĐẠI HỌC VĂN LANG · KHOA CÔNG NGHỆ THÔNG TIN

BUỔI 13 · LÝ THUYẾT

Kiểm thử phần mềm và Đảm bảo chất lượng

Kim tự tháp kiểm thử · Thiết kế test case · Test Plan · Bug Log · CI
Mở đầu Giai đoạn 3 · Phát đề bài Bài tập 3

Giảng viên chính HK261: **TS. Nguyễn Trí Hải / ThS. Nguyễn Minh Tân / ThS. Trần Công Thanh**

Học kỳ 1, năm học 2026 – 2027

NỘI DUNG BUỔI HỌC

Năm phần, kết thúc bằng đề bài Bài tập 3

01

Kim tự tháp kiểm thử

Unit, integration, system, acceptance: mỗi tầng đo gì, chi phí bao nhiêu; cô lập phụ thuộc bằng đối tượng giả.

02

Thiết kế test case

Tám cột của một test case; bốn nguồn sinh test case; phân vùng tương đương và giá trị biên; năm loại test BT3 yêu cầu; test tốt và test vô nghĩa.

03

Test Plan, phân loại bug và Bug Log

Bốn mục của Test Plan rút gọn; Critical / Major / Minor theo tiêu chí khách quan; mục Bug Log bảy dòng; trung thực trong Test Report.

04

CI: chạy test tự động mỗi lần push

GitHub Actions trên máy sạch; tệp cấu hình tối thiểu; đọc kết quả khi CI đỏ.

05

Đề bài Bài tập 3 và phần chuyên ngành

Sáu thành phần, cấu trúc PDF, bảng tiêu chí kỹ thuật; công cụ kiểm thử theo track trong tài liệu tự học.

MỤC TIÊU

Mục tiêu buổi học

1 Chọn tầng kiểm thử hợp lý

- Giải thích kim tự tháp và "hình nón ngược"
- Cô lập service khỏi CSDL và thời gian bằng đối tượng giả

2 Thiết kế test case có phương pháp

- Viết test case đủ tám cột từ tiêu chí chấp nhận
- Chọn dữ liệu thử bằng phân vùng và giá trị biên

3 Lập kế hoạch và ghi nhận lỗi

- Viết Test Plan bốn mục, có tiêu chí đo được
- Phân loại bug khách quan; ghi Bug Log bảy dòng

4 Tự động hoá kiểm thử

- Cấu hình CI chạy test mỗi lần push
- Đọc và xử lý khi CI đỏ

Bốn mục tiêu này là đầu vào trực tiếp cho thành phần 1, 2, 3 của Bài tập 3 (Test Plan, Test Report, Bug Log) — làm ở workshop buổi 14, nộp cuối buổi 16.

ĐẶT VẤN ĐỀ

Kiểm thử không phải để chứng minh phần mềm đúng. Kiểm thử là để tìm ra chỗ nó sai — trước khi người dùng tìm ra.

- Không thể chứng minh một hệ thống không còn lỗi: số trường hợp đầu vào là vô hạn. Mục tiêu thực tế là tìm được **nhiều lỗi nhất với công sức ít nhất**.
- Một buổi kiểm thử không tìm ra lỗi nào **không phải** tin tốt — thường nó có nghĩa là test quá dễ. Test case tốt là test case **có khả năng thất bại**.
- Trong học phần này, Test Report ghi trung thực 9 PASS và 3 FAIL được đánh giá **cao hơn** một Test Report toàn PASS mà kết quả không thật. BT3 yêu cầu test case **có kết quả thực tế**, không yêu cầu tất cả phải PASS.
- Phần lớn test case của em **đã viết sẵn ở BT1**: chúng chính là tiêu chí chấp nhận Given–When–Then và các luồng ngoại lệ em đã đặc tả.

01 Kim tự tháp kiểm thử

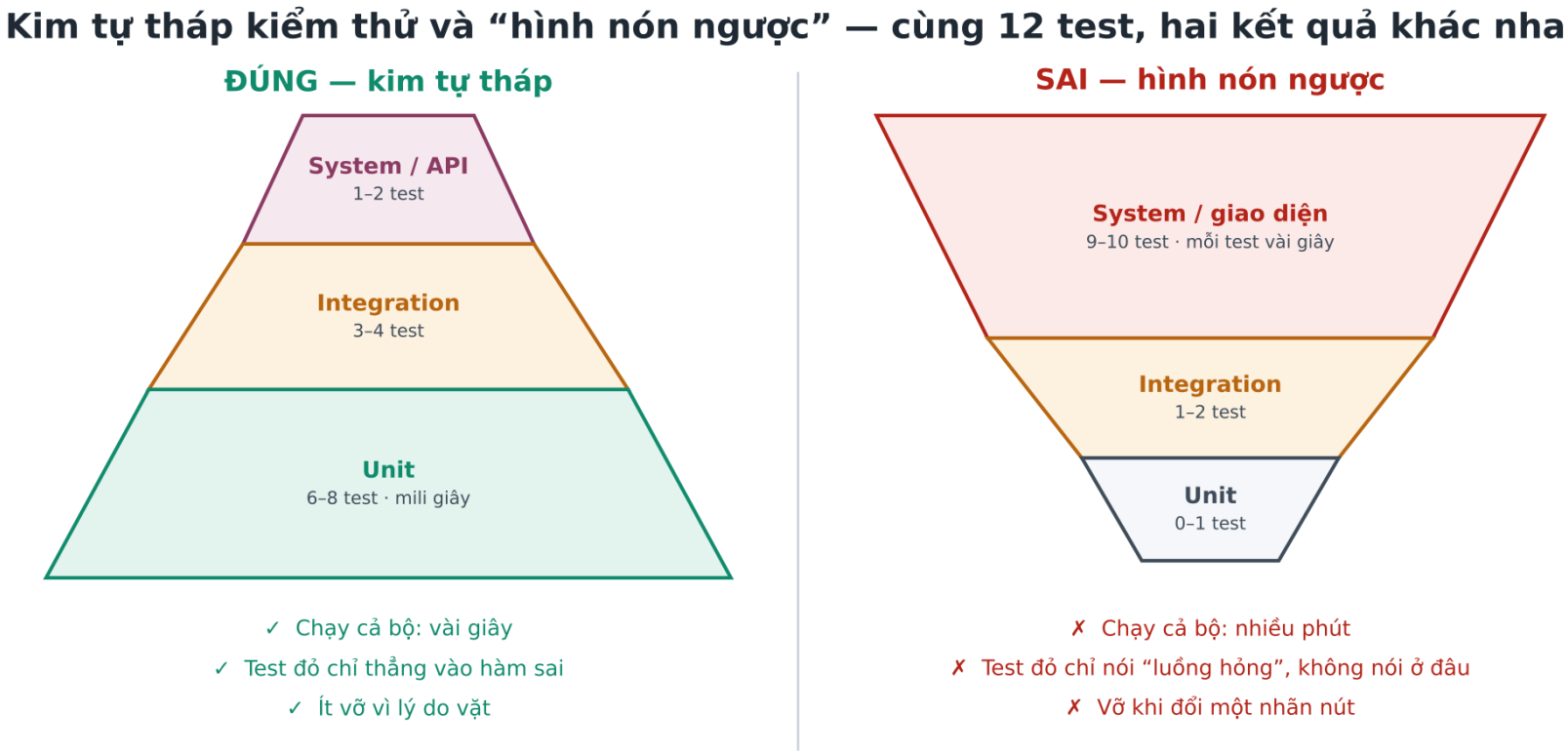
Bốn tầng · mỗi tầng đo gì · cô lập phụ thuộc bằng đối tượng giả

Bốn tầng: đo gì, chi phí bao nhiêu, cần bao nhiêu ở BT3

Tầng	Kiểm cái gì	Chi phí	Cần bao nhiêu ở BT3
Unit (đơn vị)	Một hàm hoặc một lớp, tách khỏi CSDL và mạng. Chủ yếu kiểm **quy tắc nghiệp vụ** (QT-04, QT-05…).	Rẻ, chạy trong mili giây	Nhiều nhất — 6–8 test. Nền của kim tự tháp.
Integration (tích hợp)	Hai hoặc nhiều thành phần thật làm việc với nhau: service gọi repository thật, có CSDL.	Trung bình, cần dựng và dọn dữ liệu thử	2–4 test cho các luồng chính.
System / API (hệ thống)	Toàn bộ luồng nghiệp vụ từ đầu tới cuối, qua API hoặc giao diện thật.	Đắt, chậm, dễ vỡ khi giao diện đổi	1–3 test cho luồng quan trọng nhất.
Acceptance (chấp nhận)	Người dùng thật xác nhận hệ thống đáp ứng nhu cầu của họ.	Rất đắt, cần người dùng	Không yêu cầu ở CĐTN1 — thuộc Chuyên đề tốt nghiệp 2.

BT3 yêu cầu **10–12 test case** phân bố theo hình kim tự tháp: đáy rộng (unit), đỉnh hẹp (system). Con số quan trọng hơn tổng số: **ít nhất một test cho MỖI quy tắc nghiệp vụ** em đã hiện thực.

Cùng 12 test, hai bộ test rất khác nhau



Acceptance (người dùng thật xác nhận) thuộc Chuyên đề tốt nghiệp 2 — không yêu cầu ở Bài tập 3.

Đọc hình thế nào

- Bên trái: nhiều unit test rẻ, ít system test đắt — cả bộ chạy trong vài giây.
- Bên phải: gần như chỉ có test qua giao diện — chậm, hay vỡ, và không chỉ ra lỗi ở đâu.
- Hình nón ngược thường xuất hiện khi chỉ “thử bằng tay như người dùng”.
- Câu hỏi tự kiểm: bộ test của em đang là hình nào?

VÍ DỤ — CÙNG MỘT QUY TẮC, HAI TẦNG KIỂM THỬ

QT-04 hạn cam kết: unit test và integration test

tests/han-cam-ket.test.js — Jest [SE] · pytest tương tự cho DA/AI

```
// ---------- UNIT: hàm thuần, không CSDL, không mạng ----------
const { tinhHanCamKet } = require('../src/services/han-cam-ket');

test('TC-01 QT-04 mức CAO: 24 giờ làm việc', () => {
  const nhan = new Date('2026-11-06T09:00:00+07:00');   // thứ Sáu
  expect(tinhHanCamKet(nhan, 'CAO').toISOString())
    .toBe(new Date('2026-11-07T09:00:00+07:00').toISOString());
});

test('TC-02 QT-04 biên: nhận thứ Bảy, mức CAO -> bỏ qua Chủ nhật', () => {
  const nhan = new Date('2026-11-07T15:00:00+07:00');   // thứ Bảy
  expect(tinhHanCamKet(nhan, 'CAO').getDay()).toBe(1);   // thứ Hai
});

// ---------- INTEGRATION: service + repository + CSDL thử thật ----------
const { taoTicketService } = require('../src/services/ticket.service');
const { resetDb, seedDemo, repo } = require('./helpers/db');

beforeEach(async () => { await resetDb(); await seedDemo(); });

test('TC-07 tạo phiếu mức THAP -> due_date lưu đúng trong CSDL', async () => {
  const svc = taoTicketService(repo);
  const phieu = await svc.tao({ customerId: 1, serialNo: 'SN10293A',
                                mucDo: 'THAP', moTaLoi: 'Máy sạc không vào điện' });
  const dong = await repo.timPhieu(phieu.ticketId);
  expect(dong.due_date).toEqual(tinhHanCamKet(dong.received_at, 'THAP'));
});
```

Ghi nhớ

- Unit test kiểm **quy tắc** (QT-04) trên hàm thuần — rẻ, chạy trong mili giây.
- Integration test kiểm **dữ liệu thật được lưu đúng** qua repository và CSDL thử.
- TC-02 là test biên: QT-04 chỉ tính thứ Hai đến thứ Bảy.
- Tiền điều kiện dựng bằng code (resetDb, seedDemo), không dựa vào dữ liệu có sẵn trên máy em.

Thời gian và CSDL phải được tiêm vào, không gọi thẳng

✗ Phụ thuộc ẩn — test chạy hôm nay PASS, mai FAIL

```
// service gọi thẳng đồng hồ hệ thống và CSDL
async function conBaoHanh(serialNo) {
  const tb = await db.query(
    'SELECT ngay_mua, so_thang FROM devices WHERE serial=$1',
    [serialNo]);
  const homNay = new Date();          // ← thay đổi mỗi ngày
  return thangGiua(tb.ngay_mua, homNay) <= tb.so_thang;
}
// Test phụ thuộc: CSDL đang chạy, dữ liệu có sẵn,
// và NGÀY CHẠY TEST. Sau 2027-03-01 test này đổi kết quả.
```

✓ Tiêm phụ thuộc — test xác định, chạy ở đâu cũng vậy

```
// service nhận repo và đồng hồ từ bên ngoài
function taoBaoHanhService(repo, dongHo = () => new Date()) {
  return {
    async conBaoHanh(serialNo) {
      const tb = await repo.timThietBi(serialNo);
      return thangGiua(tb.ngay_mua, dongHo()) <= tb.so_thang;
    },
  };
}
// Trong test: repo giả + đồng hồ cố định
const repoGia = { timThietBi: async () =>
  ({ ngay_mua: '2026-03-01', so_thang: 12 }) };
const svc = taoBaoHanhService(repoGia, () => new Date('2027-03-01'));
expect(await svc.conBaoHanh('SN10293A')).toBe(true);   // biên QT-05
```

Hậu quả: test "nhấp nháy" (flaky) — đỏ xanh thất thường, người chấm chạy lại không ra kết quả em ghi trong Test Report.

Nguyên tắc: thứ gì thay đổi ngoài tầm kiểm soát (thời gian, CSDL, mạng, số ngẫu nhiên) đều phải **tiêm vào** được. [DA] cố định ngày chạy bằng tham số · [AI] cố định random_state.

02 Thiết kế test case

Tám cột · bốn nguồn sinh test case · phân vùng tương đương và giá trị biên · năm loại test của BT3

TÁM CỘT CỦA MỘT TEST CASE – ĐÚNG MẪU ĐỀ BÀI BT3

Mỗi test case trong Test Report đủ tám cột

Cột	Ghi gì	Ví dụ — TC-05 (QT-03)
1. Mã	Mã duy nhất để Bug Log tham chiếu lại	TC-05
2. Tiền điều kiện	Trạng thái dữ liệu và hệ thống TRƯỚC khi chạy	Thiết bị SN10293A đang có phiếu BH-000231/2026 ở trạng thái DANG_SUA
3. Các bước	Đánh số, mỗi bước một hành động	1. POST /api/tickets với serialNo của thiết bị trên
4. Dữ liệu thử	Giá trị CỤ THỂ, không phải “một số hợp lệ”	{ customerId: 1024, serialNo: "SN10293A", mucDo: "CAO", moTaLoi: "Máy sạc không vào" }
5. Kết quả mong đợi	Quan sát được: mã trạng thái, thông điệp, VÀ tác dụng phụ	409, error = PHIEU_DANG_MO; bảng tickets KHÔNG có bản ghi mới
6. Mức ưu tiên	Cao / Trung bình / Thấp	Cao (quy tắc nghiệp vụ cốt lõi)
7. Kết quả thực tế	Điền khi CHẠY THẬT, bằng lời cụ thể	409, PHIEU_DANG_MO; đếm bảng tickets trước/sau: 120 → 120
8. PASS / FAIL	Khớp hoàn toàn mới là PASS	PASS — kèm ảnh chụp màn hình kết quả

*Kết quả mong đợi phải kiểm cả **tác dụng phụ** (“không có bản ghi mới”): hệ thống có thể trả lỗi đúng nhưng vẫn ghi một bản ghi hỏng vào CSDL.*

BỐN NGUỒN SINH TEST CASE – PHẦN LỚN ĐÃ CÓ SẴN TỪ BT1

Test case của em đến từ đâu

1

Tiêu chí chấp nhận Given–When–Then ở BT1

GIVEN thành tiền điều kiện và dữ liệu thử, WHEN thành các bước, THEN thành kết quả mong đợi. 3 story MUST × 3 tiêu chí = **9 test case có sẵn**.

2

Luồng ngoại lệ trong đặc tả use case ở BT1

Mỗi luồng ngoại lệ (3a, 4a…) là một test case — nhóm có **khả năng tìm ra lỗi cao nhất**, vì đường ngoại lệ ít được chạy nên ít được sửa.

3

Quy tắc nghiệp vụ trong case study (QT-01 … QT-15)

Mỗi quy tắc liên quan tới luồng của em sinh ra ít nhất một test tuân thủ và một test vi phạm.

4

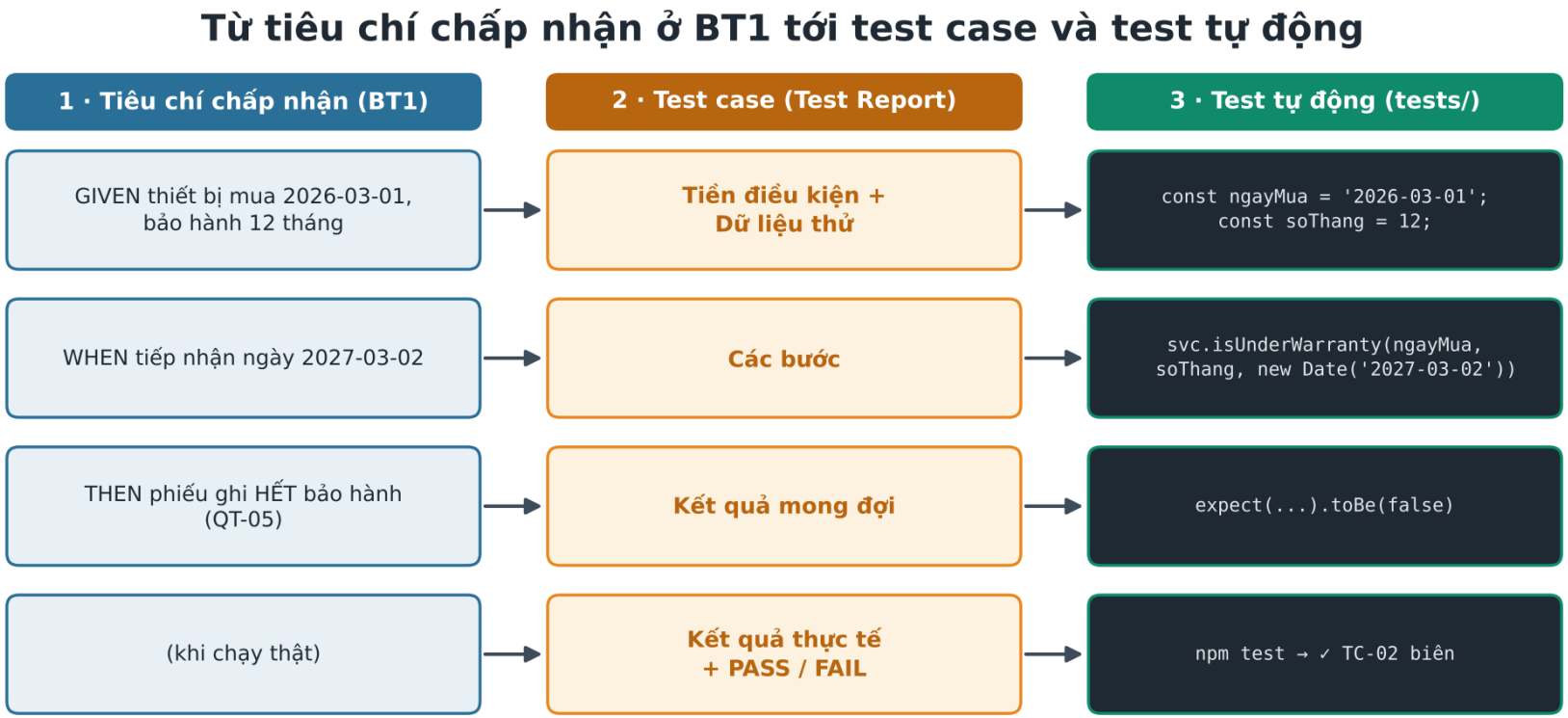
Mỗi bug em đã gặp trong Giai đoạn 2

Mỗi lỗi đã sửa nên có một test hồi quy bảo đảm nó không quay lại. Nếu em đã ghi **docs/bug-log.md** từ buổi 11, nguồn này đã có sẵn.

Làm ngay tại lớp

- Mở lại SRS và đặc tả use case của em.
- Đếm số tiêu chí chấp nhận và số luồng ngoại lệ — đó là số test case đã có.
- Liệt kê các QT-xx mà luồng của em chạm tới.
- Thiếu bao nhiêu để đủ 10–12? Bù bằng test biên và test dữ liệu sai.

Given–When–Then chuyển thẳng thành test case



3 story MUST × 3 tiêu chí = 9 test case đã có sẵn. Thêm luồng ngoại lệ và giá trị biên là đủ 10–12.

Đọc hình thế nào

- Cột 1: tiêu chí chấp nhận đã viết ở BT1 cho QT-05.
- Cột 2: mỗi dòng rơi vào đúng một cột của test case.
- Cột 3: cùng nội dung đó viết thành test tự động.
- Hai cột cuối (kết quả thực tế, PASS/FAIL) chỉ điền khi CHẠY THẬT.

Phân vùng tương đương và giá trị biên

PHÂN VÙNG TƯƠNG ĐƯƠNG

- Chia miền đầu vào thành các nhóm mà hệ thống xử lý **giống nhau**; thử **một giá trị đại diện** mỗi nhóm.
- Ví dụ moTaLoi dài 10–2000 ký tự: ba vùng — dưới 10, 10–2000, trên 2000. Ba giá trị là đủ, không cần thử 100 giá trị.
- [DA] vùng: dòng đúng định dạng · ngày sai định dạng · ô bắt buộc trống.
- [AI] vùng: mô tả bình thường · mô tả rỗng · mô tả chỉ có ký tự đặc biệt.

GIÁ TRỊ BIÊN

- Lỗi tập trung ở **ranh giới** giữa các vùng — thử ngay tại và sát ranh giới.
- Ràng buộc 10–2000: thử **9, 10, 2000, 2001**. Bắt được lỗi so sánh nhầm dấu (< thay vì ≤).
- QT-05 (≤ 12 tháng), mua 2026-03-01: thử 2027-02-28 (còn), **2027-03-01 (biên, vẫn còn)**, 2027-03-02 (hết).
- BT3 yêu cầu **2 test biên ở hai bên của cùng một ranh giới**.

Ghi rõ trong Test Plan rằng em đã dùng hai kỹ thuật này — đó là bằng chứng kiểm thử **có phương pháp**, không thử bừa.

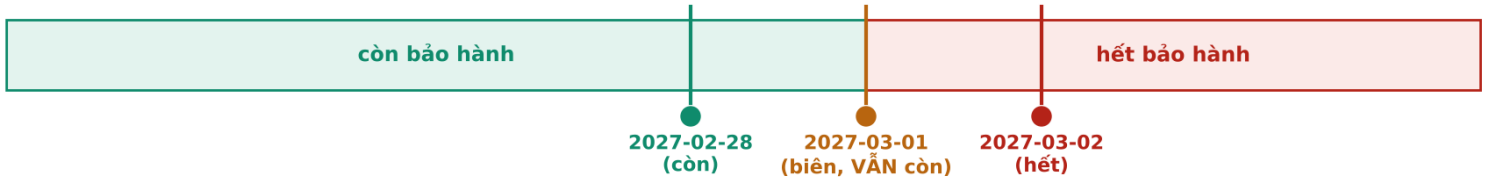
Bốn giá trị đúng chỗ tốt hơn trăm giá trị ngẫu nhiên

Phân vùng tương đương và giá trị biên — chọn ít giá trị nhưng đúng chỗ

Trường issueDesc: độ dài hợp lệ 10–2000 ký tự



QT-05: còn bảo hành nếu (ngày tiếp nhận − ngày mua) ≤ 12 tháng · mua 2026-03-01



Lỗi hay gặp nhất là so sánh nhầm dấu (< thay vì ≤). Chỉ test đặt NGAY tại và sát ranh giới mới bắt được.

Đọc hình thế nào

- Hàng trên: ba vùng của độ dài mô tả; bốn điểm nằm sát hai ranh giới.
- Hàng dưới: QT-05 — ngày 2027-03-01 là biên và theo quy tắc "≤" thì VẪN còn bảo hành.
- Điểm xanh: kỳ vọng hợp lệ / còn hạn. Điểm đỏ: kỳ vọng bị từ chối / hết hạn.
- Mỗi điểm là một dòng dữ liệu thử trong Test Report.

NĂM LOẠI TEST BT3 YÊU CẦU

10–12 test case phải đủ năm loại này

Loại	Số lượng tối thiểu	[SE]	[DA]	[AI]
1. Luồng đúng	≥ 1 cho MỖI quy tắc đã hiện thực	Thiết bị còn bảo hành → tạo được phiếu	Tệp nguồn đúng → nạp đủ số dòng	Mô tả hợp lệ → top 3 + độ tin cậy
2. Giá trị biên	2, ở hai bên cùng một ranh giới	Ngày cuối còn bảo hành / ngày đầu hết	Dòng đúng bằng ngưỡng cho phép	Mô tả đúng độ dài tối thiểu / ngắn hơn 1
3. Dữ liệu sai	2 — kiểm mã trạng thái VÀ thông báo	Thiếu trường → 400 nêu tên trường	Ngày sai → vào reject kèm lý do	Mô tả rỗng → 422, không sập
4. Bảo mật / chất lượng dữ liệu	1	Sửa phiếu của người khác → 403 (A01)	Một phép đo của ba chiều chất lượng	Test chống rò rỉ dữ liệu
5. Tính tái lập	1 — bắt buộc cả ba track	Chạy migration + seed lần hai không lỗi	Chạy pipeline lần hai → không nhân đôi	Huấn luyện lại cùng seed → cùng chỉ số

Ba test cùng một loại chỉ tính là một. Thiếu loại nào thì mục kiểm chứng **C2 (CỐT LÕI)** của BT3 không đạt.

Một test PASS chưa chắc đã kiểm được gì

✗ Test vô nghĩa — luôn PASS, không bắt được lỗi

```
test('tạo phiếu', async () => {
  const res = await request(app).post('/api/tickets')
                                .send(duLieuMau);
  expect(res).toBeDefined();          // ← luôn đúng
});

test('tính hạn cam kết', () => {
  const kq = tinhHanCamKet(nhan, 'CAO');
  expect(kq).toEqual(tinhHanCamKet(nhan, 'CAO'));
  // ← so sánh hàm với chính nó: sai cũng PASS
});

test('test 2 dùng phiếu do test 1 tạo', ...)  // phụ thuộc thứ tự
```

✓ Test có khả năng thất bại

```
test('TC-04 tạo phiếu -> 201, mã phiếu đúng định dạng', async () => {
  const res = await request(app).post('/api/tickets')
                                .send(duLieuMau);
  expect(res.status).toBe(201);
  expect(res.body.data.ticketCode)
    .toMatch(/^BH-\d{6}\/\d{4}$/);
});

test('TC-01 QT-04 mức CAO = +24 giờ làm việc', () => {
  const nhan = new Date('2026-11-06T09:00:00+07:00');
  expect(tinhHanCamKet(nhan, 'CAO'))
    .toEqual(new Date('2026-11-07T09:00:00+07:00'));  // giá trị tính
tay
});
```

Dấu hiệu test vô nghĩa: không có assert cụ thể · kỳ vọng tính bằng chính hàm cần kiểm · test phụ thuộc thứ tự chạy · test chưa bao giờ FAIL.

Phép thử nhanh: cố tình làm sai mã (đổi ≤ thành <) rồi chạy lại. Test tốt phải chuyển sang **FAIL**. Không FAIL thì test không kiểm gì cả.

03

Test Plan, phân loại bug và Bug Log

Bốn mục của Test Plan · ba mức bug · mục Bug Log bảy dòng · trung thực trong Test Report

TEST PLAN RÚT GỌN – THÀNH PHẦN 1 CỦA BT3, KHOẢNG MỘT TRANG

Bốn mục của một Test Plan rút gọn

1

Phạm vi — kiểm gì và CHỦ Ý không kiểm gì

Liệt kê luồng sẽ kiểm. Ghi rõ phần không kiểm và lý do (ví dụ: “không kiểm hiệu năng — thuộc CĐTN2”).

2

Loại test và số lượng mỗi tầng

Ví dụ: 7 unit · 2 integration · 3 API. Nêu rõ kỹ thuật chọn dữ liệu đã dùng (phân vùng, giá trị biên).

3

Tiêu chí thành công ĐO ĐƯỢC

Ví dụ: “Toàn bộ 12 test PASS · mỗi quy tắc đã hiện thực có ≥ 1 test · không còn bug Critical đang mở.” Đặt TRƯỚC khi chạy test.

4

Môi trường và dữ liệu thử

Chạy ở đâu (container của docker-compose), dữ liệu từ đâu (seeds/test_fixtures.sql), dựng và dọn thế nào (resetDb() trước mỗi test).

Câu hỏi Test Plan phải trả lời

- “Làm sao em biết em đã kiểm thử ĐỦ?”
- **Yếu:** “Em đã kiểm hết các chức năng chính.”
- **Tốt:** “Bốn quy tắc đã hiện thực, mỗi quy tắc có một test luồng đúng và một test biên; thêm hai test dữ liệu sai và ba test API. Cố ý không kiểm màn hình cấu hình vì không có quy tắc nghiệp vụ.”

VÍ DỤ — MỘT TEST PLAN RÚT GỌN ĐÃ ĐIỀN

docs/test-plan.md cho luồng tiếp nhận phiếu bảo hành

docs/test-plan.md — khoảng một trang, đặt trong repo và chép vào mục 1 của PDF

```
# Test Plan — Luồng L2: Tiếp nhận thiết bị bảo hành (track SE)

## 1. Phạm vi
Kiểm: tiếp nhận phiếu, kiểm bảo hành QT-05, sinh hạn cam kết QT-04,
      chặn phiếu trùng QT-03, phân quyền theo chủ sở hữu.
KHÔNG kiểm: màn hình cấu hình (chỉ đọc, không có quy tắc nghiệp vụ);
      hiệu năng và tải (thuộc Chuyên đề tốt nghiệp 2).

## 2. Loại test và số lượng
7 unit (service) · 2 integration (service + PostgreSQL thử) · 3 API (Supertest)
Kỹ thuật chọn dữ liệu: phân vùng tương đương; giá trị biên cho QT-04, QT-05.
Đủ 5 loại BT3: luồng đúng ×4 · biên ×2 · dữ liệu sai ×2 · bảo mật ×1 · tái lập ×1.

## 3. Tiêu chí thành công
- 12/12 test case có kết quả thực tế; mọi QT đã hiện thực có >= 1 test.
- Không còn bug Critical đang mở; bug Major còn lại có lý do và hướng xử lý.

## 4. Môi trường và dữ liệu thử
docker compose up -d --build · dữ liệu: seeds/test_fixtures.sql (12 khách,
20 thiết bị, 2 tài khoản A/B) · resetDb() chạy trước MỖI test · lệnh: npm test
```

Ghi nhớ

- Mục 1 có cả phần **cố ý không kiểm** kèm lý do.
- Mục 2 khớp đủ **năm loại** BT3 yêu cầu.
- Mục 3 phát biểu bằng **số**, đặt trước khi chạy.
- Mục 4 đủ để người khác dựng lại môi trường thử.
- [DA] thay mục 2 bằng: unit hàm làm sạch, phép đo ba chiều, chạy lại pipeline. [AI]: test hàm biến đổi, rò rỉ, tái lập, dịch vụ dự đoán.

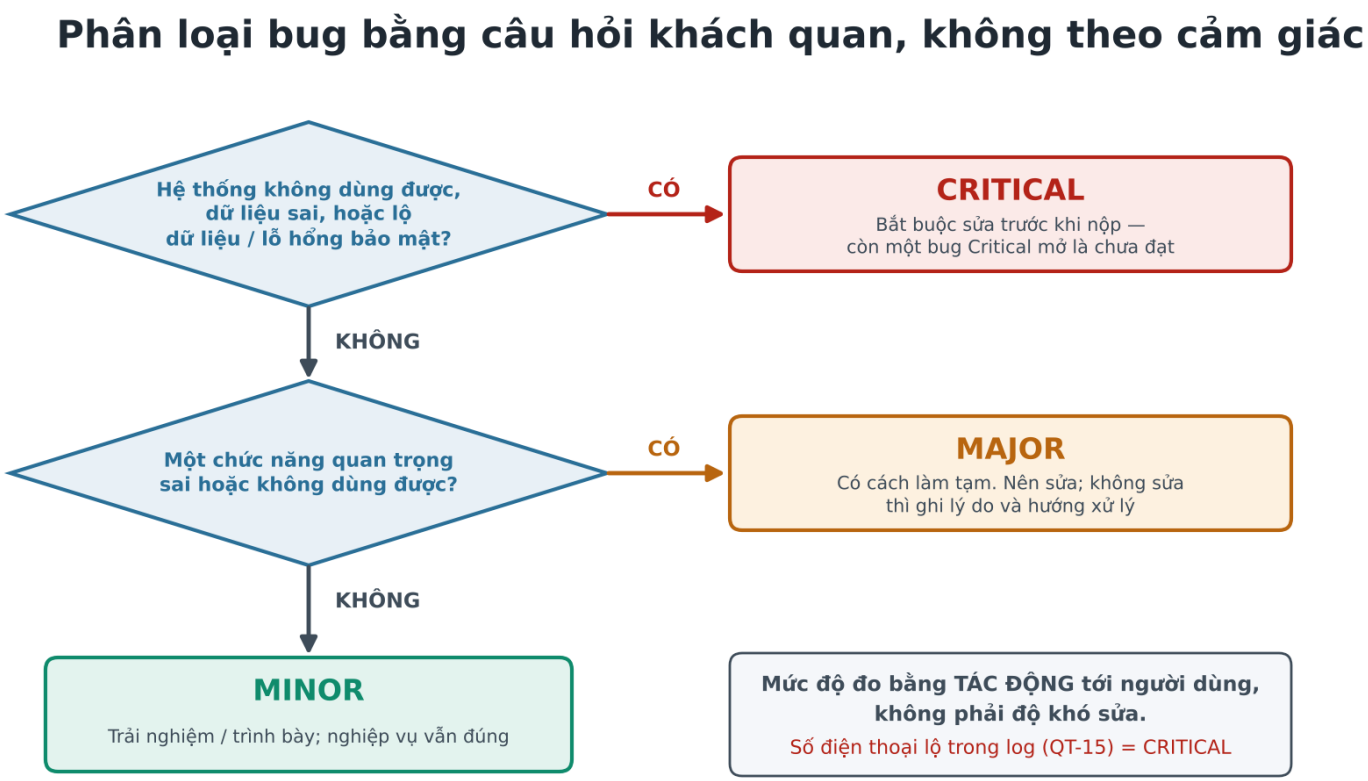
PHÂN LOẠI BUG – TIÊU CHÍ KHÁCH QUAN, KHÔNG THEO CẢM TÍNH

Ba mức bug theo đề bài BT3

Mức	Định nghĩa (đề BT3)	Ví dụ trong luồng Smart CRM	Yêu cầu ở BT3
CRITICAL	Hệ thống không dùng được, HOẶC dữ liệu bị sai, HOẶC có lỗ hổng bảo mật	Không tạo được phiếu. Sửa được phiếu của người khác. Số điện thoại đầy đủ hiện cho vai trò không được phép (QT-15).	Phải sửa hết — còn một bug Critical mở là chưa đạt tiêu chí thành công
MAJOR	Một chức năng quan trọng sai hoặc không dùng được, nhưng có cách làm khác	Bộ lọc theo trạng thái trả sai kết quả. Hạn cam kết tính sai với mức THAP.	Nên sửa; không sửa thì ghi rõ lý do và hướng xử lý
MINOR	Ảnh hưởng trải nghiệm hoặc trình bày, nghiệp vụ vẫn đúng	Nhãn sai chính tả. Cột bảng lệch. Thông báo đúng nhưng khó hiểu.	Ghi nhận là đủ; sửa được thì tốt

Mức độ đo bằng **tác động tới người dùng**, không phải độ khó sửa. Lỗi sửa trong năm phút vẫn có thể là Critical.

Hai câu hỏi, hỏi đúng thứ tự



Đọc hình thế nào

- Câu 1 trước: không dùng được, sai dữ liệu hay lộ dữ liệu → Critical.
- Câu 2 sau: chức năng quan trọng sai nhưng có cách làm khác → Major.
- Còn lại là Minor.
- Khi giảng viên hỏi “vì sao Major?”, trả lời bằng đúng hai câu này.

Một mục Bug Log đủ bảy dòng

docs/bug-log.md — ghi NGAY khi gỡ xong, đừng để tới lúc nộp mới nhớ lại

```
## BUG-03 — Tạo phiếu thất bại khi khách thiếu họ tên

1. Mã và mức độ  : BUG-03 · MAJOR — một nhóm khách không tạo được phiếu
2. Phát hiện bởi : TC-07 (Test Report)
3. Triệu chứng   : POST /api/tickets trả 500 với khách id=1024;
                   giao diện hiện "Có lỗi xảy ra", không tạo được phiếu.
4. Cách tái lập  : POST /api/tickets với customerId=1024 (full_name rỗng
                   trong dữ liệu mẫu) — lỗi xuất hiện 5/5 lần.
5. Nguyên nhân   : toTicketView() gọi full_name.trim() khi full_name = null;
                   gốc rễ: customers_raw.csv có bản ghi thiếu tên, lúc nạp không kiểm.
6. Cách sửa      : hàm chuyển đổi dùng '(không rõ)' khi nguồn rỗng (đúng hợp đồng);
                   thêm quy tắc kiểm chất lượng "thiếu họ tên" khi nạp.
7. Commit và     : https://github.com/<tài-khoản>/<repo>/commit/a3f9c21
   xác nhận        chạy lại TC-07 -> PASS; 12/12 test PASS; thêm TC-13 hồi quy.
```

Ghi nhớ

- Bảy dòng khớp đúng đề BT3 mục 2.3.
- Dòng 5 là **nguyên nhân**, không phải triệu chứng.
- Link commit phải **mở được** và trỏ đúng thay đổi — giảng viên bấm thử từng link.
- Bảy dòng này chính là bốn bước debug của buổi 11, ghi thành văn bản.
- BT3 cần ≥ 3 bug, có ít nhất một Critical hoặc Major.

Mỗi giả thuyết ba dòng — và đặt đúng tầng

Khi chưa biết nguyên nhân: viết giả thuyết để người khác kiểm chứng được

```
## Hiện tượng: danh sách phiếu hiện một phiếu HAI lần (chỉ khi phiếu
##             đã được chuyển kỹ thuật viên ít nhất một lần)

Giả thuyết 1 — TẦNG DỮ LIỆU
  Giả thuyết : truy vấn danh sách nối tickets với ticket_assignments
               mà không lọc bản phân công mới nhất -> mỗi lần chuyển
               kỹ thuật viên sinh thêm một dòng.
  Kiểm chứng : chạy riêng truy vấn danh sách với phiếu T-0412;
               SELECT ticket_id, COUNT(*) ... GROUP BY 1 HAVING COUNT(*)>1
               -> có dòng trả về thì giả thuyết đúng.
  Sửa ở đâu  : repository danh sách phiếu: chỉ nối bản phân công
               hiện hành; thêm test với phiếu đã chuyển 2 lần.

Giả thuyết 2 — TẦNG GIAO DIỆN
  Giả thuyết : trang gọi API danh sách hai lần (khi tải và khi đổi
               bộ lọc) rồi NỐI hai kết quả thay vì thay thế.
  Kiểm chứng : mở tab Network, tải trang một lần: thấy 2 yêu cầu
               GET /api/tickets và API trả đúng 1 phiếu -> đúng.
  Sửa ở đâu  : hàm cập nhật danh sách ở giao diện: thay, không nối.
```

Ghi nhớ

- Đây là bước 2 (khoanh vùng) của buổi 11, viết ra cho người KHÁC làm theo được.
- Bốn tầng: giao diện · nghiệp vụ · dữ liệu · hạ tầng. Hai giả thuyết cùng tầng chỉ tính là một.
- Dòng kiểm chứng phải nêu xem nhật ký nào, chạy lệnh hay truy vấn nào, và KẾT QUẢ NÀO thì kết luận giả thuyết đúng.
- Giả thuyết được xác nhận sẽ thành dòng 5 (nguyên nhân) của Bug Log.
- Đây cũng là đúng khuôn của câu 3b đề thi cuối kỳ.

TRUNG THỰC TRONG TEST REPORT

Cơ chế xác minh — em biết trước để không bị bất ngờ

Giảng viên sẽ làm gì

- Ở buổi 14 và buổi 16: chọn **ngẫu nhiên** một test case em ghi PASS và yêu cầu em **chạy lại tại chỗ**.
- Bấm vào từng link commit trong Bug Log.
- Hỏi nguyên nhân của một bug bất kỳ trong Bug Log.
- Áp dụng như nhau cho mọi sinh viên — đây là cách chấm công bằng.

Được đánh giá cao

- Ghi **FAIL trung thực**, rồi sửa và ghi Bug Log.
- Test Report có kết quả thực tế cụ thể và **ảnh chụp** kết quả chạy.
- Giải thích được nguyên nhân thật của từng bug.
- Nêu rõ phần chưa làm và lý do.

Bị xử lý

- Ghi PASS mà chạy lại thì FAIL: test case đó **không có minh chứng**.
- Xảy ra ở nhiều test case: xử lý như **vi phạm liêm chính**.
- Bug Log mô tả bug không thực sự gỡ, hoặc không giải thích được nguyên nhân.
- Link commit chết hoặc trỏ sai: mục đó không được tính.

04 CI: chạy test tự động mỗi lần push

GitHub Actions · máy sạch · tệp cấu hình tối thiểu · đọc kết quả khi CI đỏ

.github/workflows/ci.yml

.github/workflows/ci.yml — đặt đúng đường dẫn này thì CI mới chạy

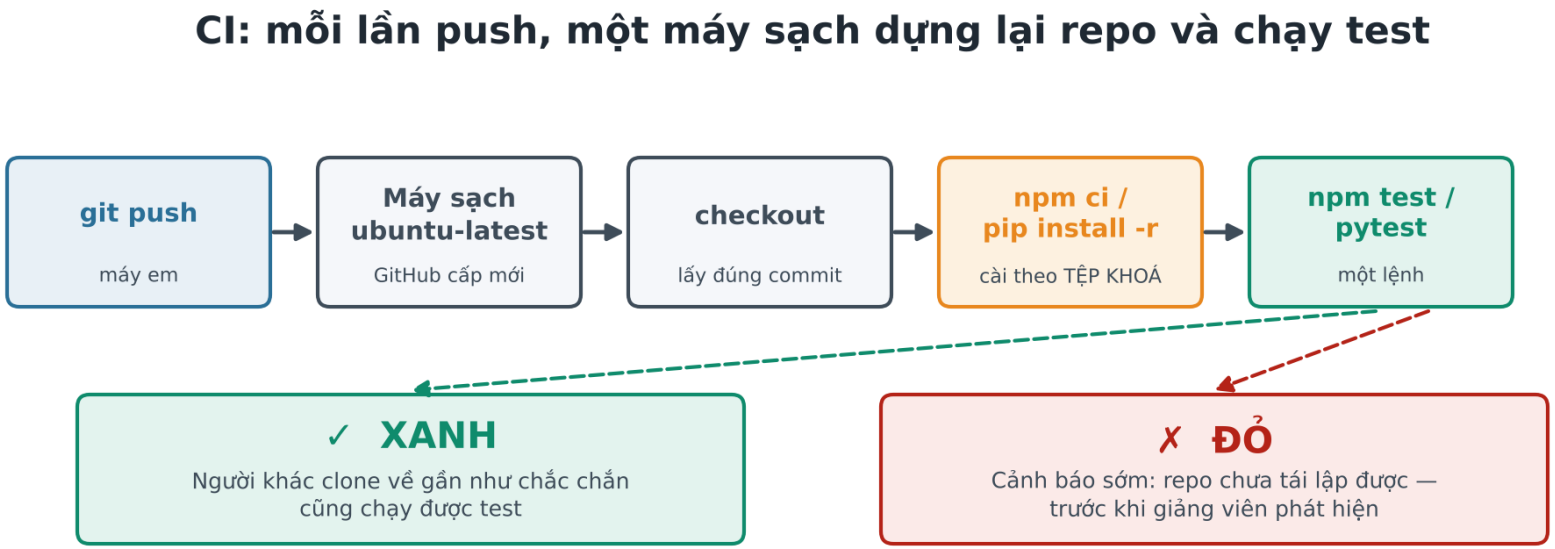
```
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      # ----- track SE (Node.js) -----
      - uses: actions/setup-node@v4
        with: { node-version: '20', cache: 'npm' }
      - run: npm ci                       # cài ĐÚNG theo package-lock.json
      - run: npm test
      # ----- track DA / AI (Python) — dùng thay cho ba bước trên -----
      # - uses: actions/setup-python@v5
      #   with: { python-version: '3.11', cache: 'pip' }
      # - run: pip install -r requirements.txt
      # - run: pytest -q
      # ----- nếu test cần PostgreSQL: khai báo dịch vụ ở cấp job -----
      #   services:
      #     db:
      #       image: postgres:16-alpine
      #       env: { POSTGRES_PASSWORD: test }
      #       ports: ['5432:5432']
```

Ghi nhớ

- Đường dẫn phải đúng: **.github/workflows/ci.yml** — sai đường dẫn thì CI không chạy.
- CI chạy trên **máy sạch** của GitHub: phép thử tự động cho tính tái lập của repo.
- Mật khẩu test trong workflow chỉ là giá trị thử; secret thật đặt trong Settings → Secrets.
- Đọc log khi đỏ: tab Actions → lần chạy thất bại → bước bị đỏ.
- BT3 **khuyến khích**, không bắt buộc CI.

MINH HOẠ — CI HOẠT ĐỘNG THẾ NÀO

Mỗi lần push là một lần "phép thử máy trắng" tự động



Ba lý do CI đỏ trong khi máy em xanh

1. Chưa commit tệp khoá → CI cài phiên bản khác với máy em.
2. Test phụ thuộc CSDL hoặc tệp dữ liệu chỉ có trên máy em, không có trong repo.
3. Đường dẫn tuyệt đối hoặc biến môi trường chưa khai báo trong workflow.

Đọc hình thế nào

- GitHub cấp một máy mới, chưa cài gì của em.
- Cài thư viện theo tệp khoá rồi chạy đúng một lệnh test.
- Xanh: người khác clone về gần như chắc chắn chạy được.
- Đỏ: tín hiệu sớm — sửa trước khi giảng viên chấm.

05

Đề bài Bài tập 3 và phần chuyên ngành

Sáu thành phần · cấu trúc PDF · bảng tiêu chí kỹ thuật · công cụ kiểm thử theo track

BÀI TẬP 3 · 10% ĐIỂM HỌC PHẦN · HẠN NỘP CUỐI BUỔI 16

Sáu thành phần phải nộp

1

Test Plan rút gọn (≈ 1 trang)

Bốn mục: phạm vi và phần cố ý không kiểm · loại test và số lượng · tiêu chí thành công đo được · môi trường và dữ liệu thử.

2

Test Report: 10–12 test case đã CHẠY THẬT

Đủ **tám cột**, đủ **năm loại**. Ghi trung thực — FAIL không bị trừ điểm; PASS khống là vấn đề liêm chính.

3

Bug Log: ≥ 3 bug đã sửa

≥ 1 Critical hoặc Major; mỗi bug đủ **bảy dòng**; link commit mở được và trỏ đúng thay đổi.

4

Gói đóng gói chạy được bằng ≤ 2 lệnh

Dockerfile + docker-compose (khuyến nghị). Từ thư mục trống, clone theo tag **v1.0**, ≤ 2 lệnh là hệ thống lên. Học ở buổi 15.

5

Deployment Guide tiếng Anh + checklist bảo mật bảy mục

Năm mục, có mục “cách kiểm tra hệ thống đã chạy đúng”; mỗi mục checklist có một dòng minh chứng. Học ở buổi 15.

6

Tài liệu kỹ thuật theo track (tiếng Anh) + khai báo AI

[SE] API docs · [DA] Pipeline doc + Data dictionary · [AI] Model card. Phụ lục khai báo AI bắt buộc.

NỘI DUNG NỘP BÀI TẬP 3 · 10% ĐIỂM HỌC PHẦN

Nộp gì cho Bài tập 3 — và nộp thế nào

1 · Hình thức và thời hạn

- Nộp qua **VLU E-learning**: **1 file PDF + link GitHub repo có tag v1.0**.
- Tên file: **BT3_<MSSV>_<Họ tên không dấu>.pdf** — một file duy nhất.
- Hạn: **trước khi kết thúc buổi 16**.
- Nộp bù: **tối đa 3 ngày**, trừ **10% điểm mỗi ngày** trễ.
- Quá 3 ngày: **0 điểm** thành phần và **không đủ điều kiện dự thi** cuối kỳ.
- Repo để **PUBLIC** — giảng viên không nhận lời mời collaborator; vì vậy không có .env hay secret.

2 · Sáu thành phần trong repo (tag v1.0)

- **TP1 – Test Plan** rút gọn, bốn mục.
- **TP2 – Test Report:** 10–12 test case đủ **tám cột**, đủ năm loại, có kết quả CHẠY THẬT.
- **TP3 – Bug Log:** ≥ 3 bug (≥ 1 Critical/Major), mỗi bug bảy dòng, link commit mở được.
- **TP4 – Gói đóng gói** dựng lại được bằng ≤ 2 lệnh từ thư mục trống.
- **TP5 – Deployment Guide** tiếng Anh (năm mục) + checklist bảo mật bảy mục.
- **TP6 – Tài liệu kỹ thuật** tiếng Anh: API docs [SE] · Pipeline doc + Data dictionary [DA] · Model card [AI].

3 · Cấu trúc file PDF

- **Trang bìa:** link repo + tag v1.0, track, mã luồng; link deploy thử nếu có.
- **Mục 1** Test Plan · **Mục 2** Test Report.
- **Mục 3** Bug Log · **Mục 4** Deployment Guide.
- **Mục 5** Checklist bảo mật · **Mục 6** Tài liệu kỹ thuật.
- **Mục 7 ẢNH CHỤP minh chứng:** test PASS · hệ thống lên được · phép thử máy trắng.
- **Phụ lục (bắt buộc):** bảng khai báo sử dụng công cụ AI.

ẢNH CHỤP KẾT QUẢ CHẠY THẬT là minh chứng bắt buộc (mục 7 của PDF): thấy rõ lệnh đã chạy và kết quả — test PASS, docker compose ps healthy, phép thử máy trắng. Test case hay mục nào không có ảnh chụp thì tính là **không có minh chứng** — chụp ngay khi chạy, đừng để tới lúc nộp.

Chín mục kiểm chứng chung, bốn mục CỐT LÕI

Mục	Kiểm chứng	Loại
C1	Gói đóng gói dựng lại được từ thư mục trống bằng ≤ 2 lệnh, theo đúng Deployment Guide	CỐT LÕI
C2	Test Report có 10–12 test case đủ tám cột, đủ năm loại	CỐT LÕI
C3	Bộ kiểm thử PASS bằng một lệnh; một test case chọn ngẫu nhiên chạy lại được tại chỗ	CỐT LÕI
C4	Bug Log ≥ 3 bug, ≥ 1 Critical/Major, đủ bảy dòng, link commit mở được và trỏ đúng	CỐT LÕI
C5	Deployment Guide đủ năm mục, có mục “cách kiểm tra hệ thống đã chạy đúng”	
C6	Checklist bảo mật bảy mục đã điền đủ, mỗi mục có minh chứng cụ thể	
C7	Không có secret nào trong repo, và .env chưa từng bị commit vào lịch sử	
C8	Chạy lại lần thứ hai (nạp dữ liệu / migration / huấn luyện) không lỗi, không nhân đôi	
C9	Tài liệu kỹ thuật theo track mô tả đúng hệ thống thật, không mô tả tính năng chưa có	

Thiếu một mục CỐT LÕI (C1–C4) → tiêu chí Chất lượng kỹ thuật về mức Yếu. Mục riêng theo track (S, D, A) xem trong đề bài.

PHẦN CHUYÊN NGÀNH – SINH VIÊN TỰ HỌC THEO TÀI LIỆU

Công cụ và kỹ thuật kiểm thử theo chuyên ngành

Tài liệu tự học buổi 13 (song ngữ Việt – Anh) có mã mẫu chạy được, mẫu Test Report tám cột và mẫu Bug Log bảy dòng cho từng track.

SE Kiểm thử API và bảo mật

- Jest/Vitest + Supertest, JUnit + MockMvc, hoặc pytest + TestClient
- Ba test cho một quy tắc: luồng đúng · biên · dữ liệu sai
- Phân biệt 400, 409, 500
- Tự kiểm năm mục OWASP Top 10:2025, ưu tiên A01
- Postman collection xuất tệp, commit vào repo

DA Ba chiều chất lượng dữ liệu

- Đầy đủ — đo theo TỪNG cột, ngưỡng riêng
- Chính xác — đối chiếu quy tắc nghiệp vụ
- Nhất quán — toàn vẹn tham chiếu, khoá duy nhất
- Báo cáo chất lượng sinh tự động
- Phép đo không đạt: bug, dữ liệu nguồn hay ngưỡng sai?

AI Đánh giá và kiểm thử mô hình

- Kiểm định chéo phân tầng, dùng Pipeline
- Báo cáo “trung bình ± độ lệch chuẩn”
- Đo thiên lệch theo ít nhất hai cách nhóm
- Bốn loại test: biến đổi · rò rỉ · tái lập · dịch vụ
- Confusion matrix → Bug Log hoặc Model card

TỰ KIỂM TRA NHANH

Năm câu hỏi trước khi rời lớp

1. **Vì sao bộ test "hình nón ngược" vừa chậm vừa khó chỉ ra lỗi nằm ở đâu?**
   Gợi ý: khi một system test đỏ, nó cho em biết những gì?

2. **Ràng buộc mô tả lỗi dài 10–2000 ký tự: bốn giá trị biên cần thử là gì, và chúng bắt được lỗi gì?**
   Gợi ý: nghĩ tới < và ≤.

3. **Kể tên năm loại test BT3 yêu cầu. Test Report của em đang thiếu loại nào?**
   Gợi ý: luồng đúng, biên, dữ liệu sai, …

4. **Hệ thống trả đúng mã 409 nhưng vẫn lưu một phiếu hỏng vào CSDL. Test case của em có phát hiện được không?**
   Gợi ý: kết quả mong đợi có kiểm tác dụng phụ không?

5. **Số điện thoại khách hiện đầy đủ trong log. Đây là bug mức nào, vì sao?**
   Gợi ý: hỏi câu 1 trong cây phân loại.

KẾT THÚC BUỔI HỌC

Chốt buổi 13 và chuẩn bị cho buổi 14

Tự học, tự nghiên cứu

- Đọc tài liệu tự học buổi 13 theo track (song ngữ).
- Viết Test Plan bốn mục cho prototype của em.
- Liệt kê 10–12 test case từ bốn nguồn; đối chiếu đủ năm loại; xác định giá trị biên.
- Hoàn thiện docs/bug-log.md theo mẫu bảy dòng.
- Chạy thử MỘT test case từ đầu tới cuối, ghi đủ tám cột và **chụp ảnh** kết quả.
- Đọc Sommerville (2015), Chapter 8 – Software Testing.

Buổi 14 (thực hành) — Workshop kiểm thử hệ thống cá nhân

- Nhận phản hồi Bài tập 2; sửa lỗi theo thứ tự ưu tiên.
- Hoàn thiện Test Plan; chạy thật 10–12 test case, ghi kết quả thực tế và chụp ảnh.
- Phân loại bug; sửa ≥ 3 bug Critical/Major, commit riêng từng bug.
- Giảng viên chọn ngẫu nhiên một test case ghi PASS để em chạy lại tại chỗ.
- Cấu hình GitHub Actions CI (khuyến khích).